

From Lab Tests to Quality Scores



From Lab Tests to Quality Scores: A Neural Network Model for Red Wine Quality at GNAWC

Seif H. Kungulio

January 17, 2026

Contents

Business Understanding	1
Background	1
Problem Statement	1
Objectives	1
Success Criteria	1
Data Understanding	2
Load Data	2
Data Dimensions	2
Initial Data Exploration	2
Data Summary	3
Missing and Duplicated Values	4
Data Cleaning	4
Data Skimming	4
Target Variable Distribution	5
Correlation Heatmap	6
Check Business Insight Features	7
Data Preparation	8
Data Splitting	8
Display Data Dimensions	8
Recipe Creation	8
Convert to Matrix Format	9
Sanity Check	9
Modeling	10
Baseline Metrics (Naive)	10
Neural Network Model (Regression)	10
Predictions and Performance	11
Diagnostic Plots	11
Evaluation and Conclusion	14
Model Training	14
Conclusion	14
References	15

Business Understanding

Background

Great North American Wine Company (GNAWC) collects physicochemical lab measurements (e.g., acidity, sugar, alcohol, sulphates, pH) during production. Final quality ratings, however, are primarily determined by sensory evaluation, which can be subjective and costly.

Problem Statement

Build a neural network model that predicts wine quality scores using physicochemical tests only, enabling earlier, standardized, data-driven screening before bottling/distribution.

Objectives

- Predict quality scores accurately from lab measurements
- Detect potential low-quality batches earlier
- Reduce routine reliance on tasting panels for screening
- Improve quality decision consistency

Success Criteria

- **Primary:** Low prediction error (MAE/RMSE) on a held-out test set
- **Operational:** Strong detection of low-quality wines (e.g., quality ≤ 5) via recall/ROC-AUC (optional classification view)
- **Business:** Practical thresholding rules for early screening (flagging likely low-quality batches)

Data Understanding

Load Data

```
wine.df <- read.csv("resources/wine.csv")
```

Data Dimensions

Get the dimensions of the dataset (number of rows and columns).

```
dim(wine.df)
```

```
## [1] 1143    13
```

Initial Data Exploration

Check the column names of the dataset.

```
colnames(wine.df)
```

```
## [1] "fixed.acidity"      "volatile.acidity"   "citric.acid"
## [4] "residual.sugar"     "chlorides"          "free.sulfur.dioxide"
## [7] "total.sulfur.dioxide" "density"            "pH"
## [10] "sulphates"          "alcohol"            "quality"
## [13] "Id"
```

View the data structure

```
glimpse(wine.df)
```

```
## Rows: 1,143
## Columns: 13
## $ fixed.acidity      <dbl> 7.4, 7.8, 7.8, 11.2, 7.4, 7.4, 7.9, 7.3, 7.8, 6.7~
## $ volatile.acidity  <dbl> 0.700, 0.880, 0.760, 0.280, 0.700, 0.660, 0.600, ~
## $ citric.acid        <dbl> 0.00, 0.00, 0.04, 0.56, 0.00, 0.00, 0.06, 0.00, 0~
## $ residual.sugar    <dbl> 1.9, 2.6, 2.3, 1.9, 1.9, 1.8, 1.6, 1.2, 2.0, 1.8,~
## $ chlorides          <dbl> 0.076, 0.098, 0.092, 0.075, 0.076, 0.075, 0.069, ~
## $ free.sulfur.dioxide <dbl> 11, 25, 15, 17, 11, 13, 15, 15, 9, 15, 16, 9, 35,~
## $ total.sulfur.dioxide <dbl> 34, 67, 54, 60, 34, 40, 59, 21, 18, 65, 59, 29, 1~
## $ density            <dbl> 0.9978, 0.9968, 0.9970, 0.9980, 0.9978, 0.9978, 0~
## $ pH                 <dbl> 3.51, 3.20, 3.26, 3.16, 3.51, 3.51, 3.30, 3.39, 3~
## $ sulphates          <dbl> 0.56, 0.68, 0.65, 0.58, 0.56, 0.56, 0.46, 0.47, 0~
## $ alcohol            <dbl> 9.4, 9.8, 9.8, 9.8, 9.4, 9.4, 9.4, 10.0, 9.5, 9.2~
## $ quality            <int> 5, 5, 5, 6, 5, 5, 5, 7, 7, 5, 5, 5, 7, 6, 5, 5, 5~
## $ Id                 <int> 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 13, 16, 19, 21~
```

Display the first 6 rows of the dataset to understand its structure and contents.

```
head(wine.df)
```

```
##   fixed.acidity volatile.acidity citric.acid residual.sugar chlorides
## 1          7.4          0.70          0.00          1.9      0.076
## 2          7.8          0.88          0.00          2.6      0.098
## 3          7.8          0.76          0.04          2.3      0.092
## 4         11.2          0.28          0.56          1.9      0.075
## 5          7.4          0.70          0.00          1.9      0.076
## 6          7.4          0.66          0.00          1.8      0.075
##   free.sulfur.dioxide total.sulfur.dioxide density   pH sulphates alcohol
## 1                11                34 0.9978 3.51    0.56    9.4
## 2                25                67 0.9968 3.20    0.68    9.8
## 3                15                54 0.9970 3.26    0.65    9.8
## 4                17                60 0.9980 3.16    0.58    9.8
## 5                11                34 0.9978 3.51    0.56    9.4
## 6                13                40 0.9978 3.51    0.56    9.4
##   quality Id
## 1         5 0
## 2         5 1
## 3         5 2
## 4         6 3
## 5         5 4
## 6         5 5
```

Data Summary

Provide a summary of the dataset to get an overview of the variables and their distributions.

```
summary(wine.df)
```

```
##   fixed.acidity   volatile.acidity   citric.acid   residual.sugar
##   Min.   : 4.600   Min.   :0.1200   Min.   :0.0000   Min.   : 0.900
##   1st Qu.: 7.100   1st Qu.:0.3925   1st Qu.:0.0900   1st Qu.: 1.900
##   Median : 7.900   Median :0.5200   Median :0.2500   Median : 2.200
##   Mean    : 8.311   Mean    :0.5313   Mean    :0.2684   Mean    : 2.532
##   3rd Qu.: 9.100   3rd Qu.:0.6400   3rd Qu.:0.4200   3rd Qu.: 2.600
##   Max.    :15.900   Max.    :1.5800   Max.    :1.0000   Max.    :15.500
##   chlorides   free.sulfur.dioxide total.sulfur.dioxide   density
##   Min.   :0.01200   Min.   : 1.00    Min.   : 6.00    Min.   :0.9901
##   1st Qu.:0.07000   1st Qu.: 7.00    1st Qu.: 21.00    1st Qu.:0.9956
##   Median :0.07900   Median :13.00    Median : 37.00    Median :0.9967
##   Mean    :0.08693   Mean    :15.62    Mean    : 45.91    Mean    :0.9967
##   3rd Qu.:0.09000   3rd Qu.:21.00    3rd Qu.: 61.00    3rd Qu.:0.9978
##   Max.    :0.61100   Max.    :68.00    Max.    :289.00    Max.    :1.0037
##   pH          sulphates          alcohol          quality
##   Min.   :2.740   Min.   :0.3300   Min.   : 8.40    Min.   :3.000
##   1st Qu.:3.205   1st Qu.:0.5500   1st Qu.: 9.50    1st Qu.:5.000
##   Median :3.310   Median :0.6200   Median :10.20    Median :6.000
##   Mean    :3.311   Mean    :0.6577   Mean    :10.44    Mean    :5.657
##   3rd Qu.:3.400   3rd Qu.:0.7300   3rd Qu.:11.10    3rd Qu.:6.000
##   Max.    :4.010   Max.    :2.0000   Max.    :14.90    Max.    :8.000
##   Id
```



```
## Min.    : 0
## 1st Qu.: 411
## Median : 794
## Mean    : 805
## 3rd Qu.:1210
## Max.    :1597
```

Missing and Duplicated Values

Check for missing values in the dataset.

```
colSums(is.na(wine.df))
```

```
##      fixed.acidity    volatile.acidity    citric.acid
##              0              0              0
##      residual.sugar      chlorides  free.sulfur.dioxide
##              0              0              0
## total.sulfur.dioxide      density              pH
##              0              0              0
##          sulphates      alcohol      quality
##              0              0              0
##              Id
##              0
```

Check for duplicated values in the dataset.

```
dupes <- wine.df[duplicated(wine.df) | duplicated(wine.df, fromLast = TRUE), ]
# Print or inspect the duplicate entries
print(dupes)
```

```
## [1] fixed.acidity    volatile.acidity    citric.acid
## [4] residual.sugar    chlorides          free.sulfur.dioxide
## [7] total.sulfur.dioxide density            pH
## [10] sulphates         alcohol            quality
## [13] Id
## <0 rows> (or 0-length row.names)
```

Data Cleaning

Clean the column names for easier handling.

```
wine.df <- wine.df %>%
  janitor::clean_names()
```

Data Skimming

Get a detailed summary of the dataset using the `skimr` package.


```
skim(wine.df)
```

Table 1: Data summary

Name	wine.df
Number of rows	1143
Number of columns	13
Column type frequency:	
numeric	13
Group variables	None

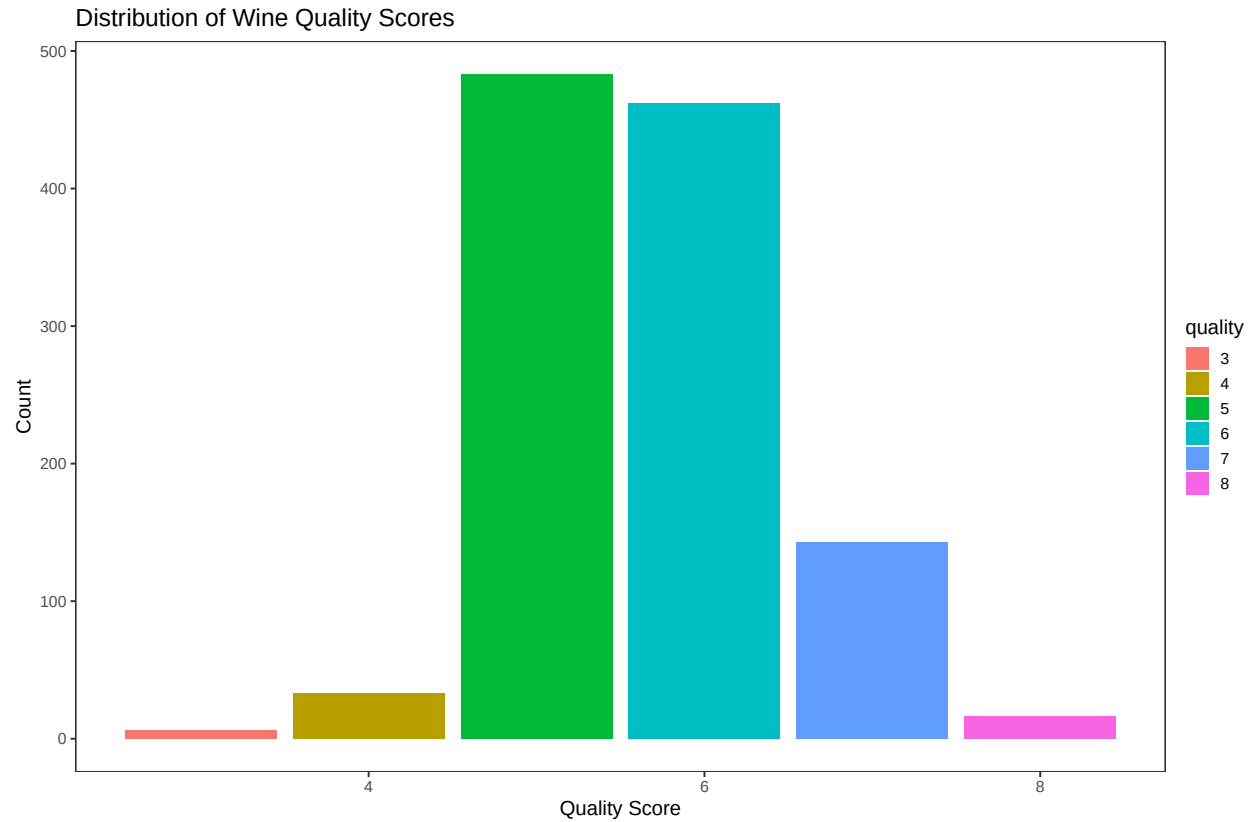
Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
fixed_acidity	0	1	8.31	1.75	4.60	7.10	7.90	9.10	15.90	
volatile_acidity	0	1	0.53	0.18	0.12	0.39	0.52	0.64	1.58	
citric_acid	0	1	0.27	0.20	0.00	0.09	0.25	0.42	1.00	
residual_sugar	0	1	2.53	1.36	0.90	1.90	2.20	2.60	15.50	
chlorides	0	1	0.09	0.05	0.01	0.07	0.08	0.09	0.61	
free_sulfur_dioxide	0	1	15.62	10.25	1.00	7.00	13.00	21.00	68.00	
total_sulfur_dioxide	0	1	45.91	32.78	6.00	21.00	37.00	61.00	289.00	
density	0	1	1.00	0.00	0.99	1.00	1.00	1.00	1.00	
p_h	0	1	3.31	0.16	2.74	3.20	3.31	3.40	4.01	
sulphates	0	1	0.66	0.17	0.33	0.55	0.62	0.73	2.00	
alcohol	0	1	10.44	1.08	8.40	9.50	10.20	11.10	14.90	
quality	0	1	5.66	0.81	3.00	5.00	6.00	6.00	8.00	
id	0	1	804.97	464.00	0.00	411.00	794.00	1209.50	1597.00	

Target Variable Distribution

Visualize the distribution of the target variable `quality`.

```
wine.df %>%
  ggplot(aes(x = quality, fill = factor(quality))) +
  geom_bar() +
  labs(title = "Distribution of Wine Quality Scores",
       x = "Quality Score",
       y = "Count",
       fill = "quality")
```



Correlation Heatmap

Visualize the correlation matrix of numeric features using a heatmap.

```
wine.df |>
  select(where(is.numeric)) |>
  cor() |>
  as.data.frame() |>
  rownames_to_column("feature") |>
  pivot_longer(-feature, names_to = "feature2", values_to = "corr") |>
  ggplot(aes(feature, feature2, fill = corr)) +
  geom_tile() +
  labs(title = "Correlation Heatmap (Numeric Features)",
       x = "",
       y = "") +
  scale_fill_viridis_c() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Check Business Insight Features

Analyze specific features for business insights.

```
wine.df |>
  group_by(quality) |>
  summarise(
    n = n(),
    alcohol_mean = mean(alcohol, na.rm = TRUE),
    volatile_acidity_mean = mean(volatile_acidity, na.rm = TRUE),
    sulphates_mean = mean(sulphates, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(quality)
```

```
## # A tibble: 6 x 5
##   quality      n alcohol_mean volatile_acidity_mean sulphates_mean
##   <int> <int>      <dbl>             <dbl>          <dbl>
## 1     3     6      9.69              0.898          0.55
## 2     4    33     10.3              0.7            0.638
## 3     5  483     9.90              0.585          0.613
## 4     6  462    10.7              0.505          0.677
## 5     7  143    11.5              0.394          0.744
## 6     8   16    11.9              0.41           0.766
```

Data Preparation

Data Splitting

Split the dataset into training and testing sets.

```
set.seed(123)
data_split <- initial_split(wine.df, prop = 0.8, strata = quality)
train_val <- training(data_split)
test_df <- testing(data_split)
```

Further split the training set into training and validation sets.

```
data_split2 <- initial_split(train_val, prop = 0.8, strata = quality)
train_df <- training(data_split2)
val_df <- testing(data_split2)
```

Display Data Dimensions

```
kable(dimensions_data <- tibble(
  Dataset = c("Full Dataset", "Train + Validation Set", "Training Set",
              "Validation Set", "Test Set"),
  Rows = c(nrow(wine.df), nrow(train_val), nrow(train_df),
            nrow(val_df), nrow(test_df)),
  Columns = c(ncol(wine.df), ncol(train_val), ncol(train_df),
              ncol(val_df), ncol(test_df))
))
```

Dataset	Rows	Columns
Full Dataset	1143	13
Train + Validation Set	913	13
Training Set	729	13
Validation Set	184	13
Test Set	230	13

Recipe Creation

Create a recipe for data preprocessing.

```
rec <- recipe(quality ~ ., data = train_df) |>
  step_zv(all_predictors()) |>
  step_normalize(all_predictors())
prep_rec <- prep(rec, training = train_df, retain = TRUE)
train_processed <- bake(prep_rec, new_data = train_df)
val_processed <- bake(prep_rec, new_data = val_df)
test_processed <- bake(prep_rec, new_data = test_df)
glimpse(train_processed)
```

```
## Rows: 729
## Columns: 13
## $ fixed_acidity      <dbl> -0.2951279, -0.2951279, -0.5254794, -0.5254794, --
## $ volatile_acidity  <dbl> 1.99468936, 1.31689245, 0.97799399, 0.75206169, 0~
## $ citric_acid       <dbl> -1.36168183, -1.15731391, -1.36168183, -1.3616818~
## $ residual_sugar    <dbl> 0.07743112, -0.14292263, -0.43672762, -0.51017887~
## $ chlorides         <dbl> 0.23234870, 0.10683391, -0.22787217, -0.24879130,~
## $ free_sulfur_dioxide <dbl> 0.95173412, -0.05884657, -0.46307885, -0.26096271~
## $ total_sulfur_dioxide <dbl> 0.6544049, 0.2611270, -0.3439158, -0.1624030, 0.4~
## $ density          <dbl> 0.06538914, 0.17029381, 0.58991250, 0.58991250, --
## $ p_h              <dbl> -0.69800832, -0.31469528, 1.28244241, 1.28244241,~
## $ sulphates         <dbl> 0.15279042, -0.03040730, -0.58000047, -0.58000047~
## $ alcohol           <dbl> -0.5929908, -0.5929908, -0.9651069, -0.9651069, --
## $ id               <dbl> -1.757352, -1.755160, -1.750775, -1.748582, -1.74~
## $ quality          <int> 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 4, 5, 5~
```

Convert to Matrix Format

Convert the processed data into matrix format for modeling.

```
x_train <- train_processed |> select(-quality) |> as.matrix()
y_train <- train_processed$quality
x_val <- val_processed |> select(-quality) |> as.matrix()
y_val <- val_processed$quality
x_test <- test_processed |> select(-quality) |> as.matrix()
y_test <- test_processed$quality
```

Sanity Check

Perform a sanity check on the processed data.

```
dim(x_train); length(y_train)
```

```
## [1] 729 12
```

```
## [1] 729
```

```
dim(x_val); length(y_val)
```

```
## [1] 184 12
```

```
## [1] 184
```

```
dim(x_test); length(y_test)
```

```
## [1] 230 12
```

```
## [1] 230
```

Modeling

Baseline Metrics (Naive)

Calculate baseline metrics using the mean quality score from the training set.

```
baseline_pred <- rep(mean(y_train), length(y_test))
baseline_tbl <- tibble(
  truth = y_test,
  estimate = baseline_pred
)

baseline_tbl %>%
  summarize(
    RMSE = rmse_vec(truth, estimate),
    MAE = mae_vec(truth, estimate)
  )
```

```
## # A tibble: 1 x 2
##   RMSE   MAE
##   <dbl> <dbl>
## 1 0.833 0.699
```

Neural Network Model (Regression)

```
# neuralnet expects a data.frame with numeric columns and a formula
train_nn <- train_processed |> as.data.frame()
val_nn   <- val_processed   |> as.data.frame()
test_nn  <- test_processed  |> as.data.frame()

set.seed(123)

# Fit a feed-forward neural network for regression
# Note: smaller hidden layers keep training stable and fast on this dataset.
nn_fit <- neuralnet::neuralnet(
  quality ~ .,
  data = train_nn,
  hidden = c(10, 5),
  linear.output = TRUE,
  lifesign = "minimal",
  stepmax = 1e6
)
```

```
## hidden: 10, 5   thresh: 0.01   rep: 1/1   steps: 139180   error: 39.28991 time: 2.88 mins
```

```
#nn_fit
```

Predictions and Performance

```
# Predict on validation and test sets
val_pred <- neuralnet::compute(nn_fit, val_nn |> dplyr::select(-quality))$net.result[, 1]
test_pred <- neuralnet::compute(nn_fit, test_nn |> dplyr::select(-quality))$net.result[, 1]

val_metrics <- tibble(truth = val_nn$quality, estimate = val_pred) |>
  summarise(
    RMSE = yardstick::rmse_vec(truth, estimate),
    MAE = yardstick::mae_vec(truth, estimate)
  )

test_metrics <- tibble(truth = test_nn$quality, estimate = test_pred) |>
  summarise(
    RMSE = yardstick::rmse_vec(truth, estimate),
    MAE = yardstick::mae_vec(truth, estimate)
  )

list(Validation = val_metrics, Test = test_metrics)
```

```
## $Validation
## # A tibble: 1 x 2
##   RMSE  MAE
##   <dbl> <dbl>
## 1 0.780 0.565
##
## $Test
## # A tibble: 1 x 2
##   RMSE  MAE
##   <dbl> <dbl>
## 1 0.792 0.546
```

Diagnostic Plots

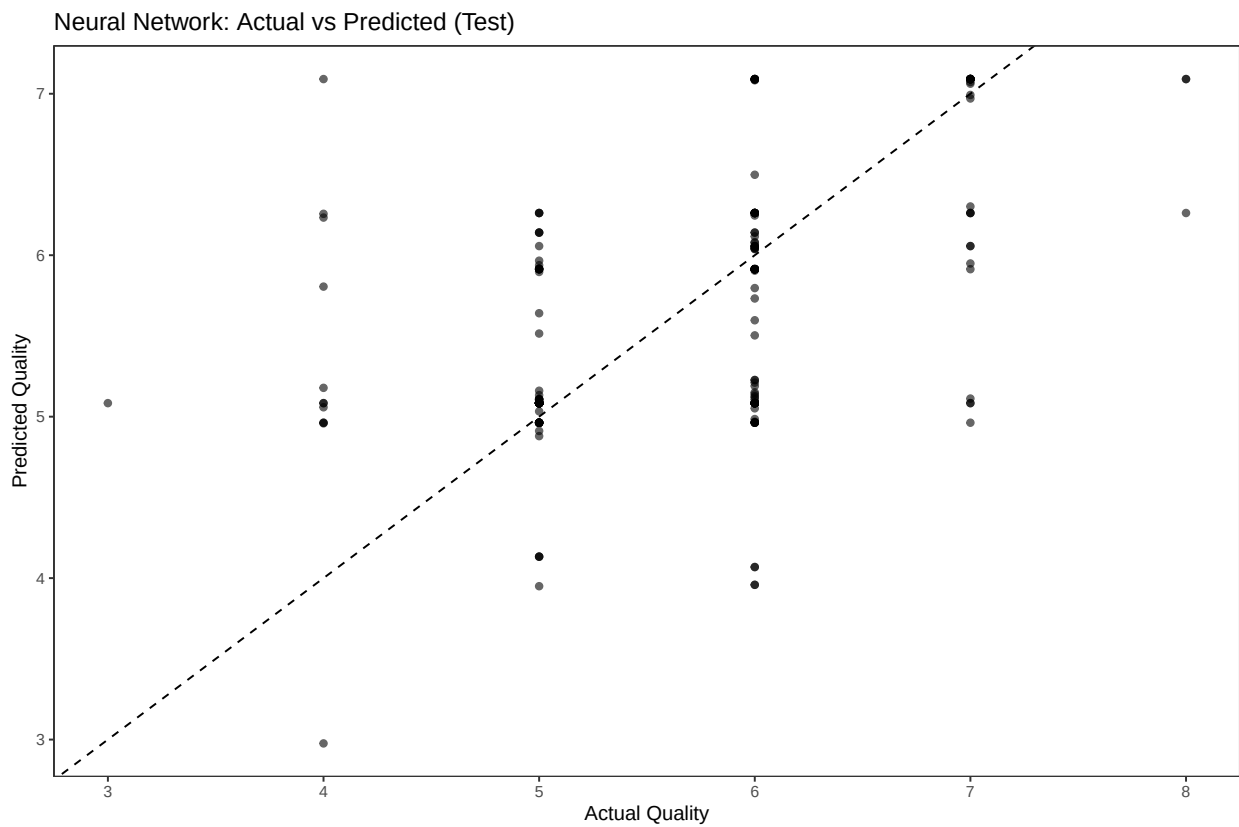
```
# Actual vs Predicted (Test)
plot_tbl <- tibble(
  truth = test_nn$quality,
  estimate = test_pred,
  residual = test_nn$quality - test_pred
)

p1 <- ggplot(plot_tbl, aes(x = truth, y = estimate)) +
  geom_point(alpha = 0.6) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed") +
  labs(
    title = "Neural Network: Actual vs Predicted (Test)",
    x = "Actual Quality",
    y = "Predicted Quality"
  )
```

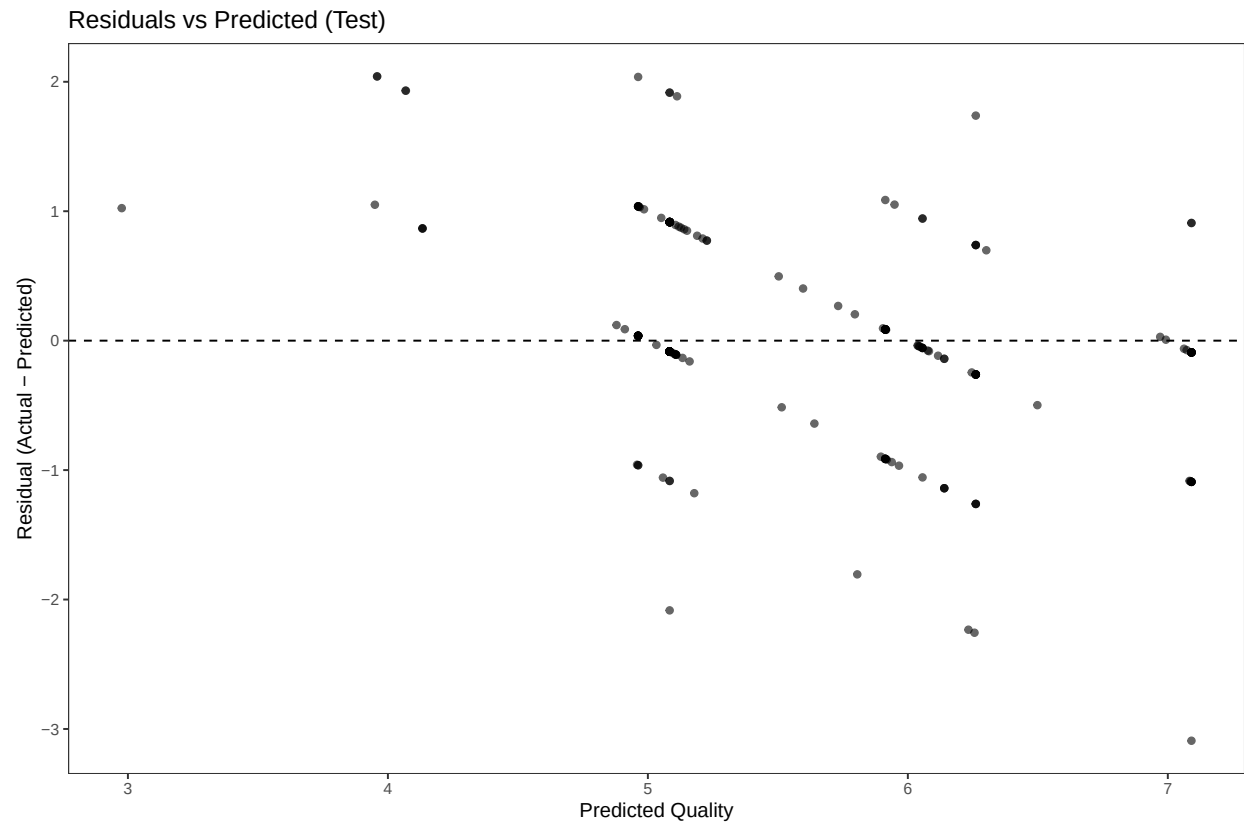


```
p2 <- ggplot(plot_tbl, aes(x = estimate, y = residual)) +  
  geom_point(alpha = 0.6) +  
  geom_hline(yintercept = 0, linetype = "dashed") +  
  labs(  
    title = "Residuals vs Predicted (Test)",  
    x = "Predicted Quality",  
    y = "Residual (Actual - Predicted)"  
  )
```

p1



p2



Evaluation and Conclusion

Model Training

Conclusion

References

1. UCI Machine Learning Repository: Wine Quality Data Set. <https://archive.ics.uci.edu/ml/datasets/wine+quality>
2. Fritsch, S., & Guenther, F. (2019). *neuralnet: Training of Neural Networks*. R package.
3. Kuhn, M., & Johnson, K. (2019). *Feature Engineering and Selection: A Practical Approach for Predictive Models*. CRC Press.
4. Wickham, H., et al. (2019). *Welcome to the tidyverse*. Journal of Open Source Software, 4(43), 1686.
5. Allaire, J., et al. (2024). *R Markdown: The Definitive Guide*. Chapman and Hall/CRC.